BCSE308L / BCSE308P – COMPUTER NETWORKS

Course Project

Predictive vs. Reactive Load Balancing in Software-Defined Networks: An Empirical Study under Flash-Crowd Traffic Conditions

Submitted in partial fulfillment of the requirements for the Course in

Bachelor of Technology

in

Computer Science and Engineering (AI and ML)

by

Fida M S (24BAI0180)

Under the Supervision of

Dr. Yoganand S

Assistant Professor Senior Grade -2

Center for Artificial Intelligence and Research (CFAIR)



October 2026

DECLARATION

I hereby declare that the project entitled “Predictive vs. Reactive Load Balancing in Software-Defined Networks: An Empirical Study under Flash-Crowd Traffic Conditions” submitted by me, for the award of the course BCSE308L in Computer Science and Engineering to VIT is a record of bonafide work carried out by me under the supervision of Prof. / Dr. Yoganand S.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other courses in this institute or any other institute or university.

Place : Vellore

Date :

Signature of the Candidate

Fida M S (24BAI0180)

CERTIFICATE

This is to certify that the project entitled “Predictive vs. Reactive Load Balancing in Software-Defined Networks: An Empirical Study under Flash-Crowd Traffic Conditions” submitted by Fida M S (24BAI0180), School of Computer Science and Engineering, VIT, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by him under my supervision during Fall Semester 2026-2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other courses in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date :

Signature of the Guide
Prof. Yoganand S

Examiner(s)

ACKNOWLEDGEMENTS

I am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering (SCOPE), for his unwavering support and encouragement. His leadership and vision have greatly inspired me to strive for excellence. The Dean's dedication to academic excellence and innovation has been a constant source of motivation for me. I appreciate his efforts in creating an environment that nurtures creativity and critical thinking.

I express my profound appreciation to Gayathri P, the Head of the Department of Computer Science and Engineering (AI&ML), for her insightful guidance and continuous support. Her expertise and advice have been crucial in shaping the direction of my project. The Head of Department's commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. Her constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project guide, Dr. Yoganand S, for his dedicated mentorship and invaluable feedback. His patience, knowledge, and encouragement have been pivotal in the successful completion of this project. My supervisor's willingness to share his expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. His support has not only contributed to the success of this project but has also enriched my overall academic experience.

Thank you all for your contributions and support.

Fida M S
24BAI0180

TABLE OF CONTENTS

List of Figures **7**

List of Tables **8**

List of Abbreviations **9**

Symbols and Notations **10**

Abstract **11**

Chapter 1. Introduction **12**

1.1 Introduction 12

1.2 Problem Statement 12

1.3 Objectives 12

1.4 Scope and Assumptions 12

1.5 Report Structure 13

Chapter 2. Background and Related Work **14**

2.1 Classic Load Balancing within SDN 14

2.2 Predictive & ML Based Balancing 14

2.3 EWMA / Peak-EWMA Latency Aware 14

2.4 Flash Crowds & Bursty Traffic 14

2.5 Should We Be Using Load? 14

Chapter 3. System Design and Methodology **16**

3.1 Architecture & Load Indicator 16

3.2 Load Balancing Policies 16

3.3 Testbed 17

3.4 Workload & Backend 18

3.5 Metrics & Method 18

Chapter 4. Implementation **19**

4.1 Tools & Environment 19

4.2 Controller & Rules 19

4.3 Bug with First Packet 19

4.4 Indicators 19

4.5 Controller Handlers 19

4.6 Packet Flow 20
Chapter 5. Results and Discussion 22
5.1 Predictor Performance 22
5.2 Homogeneous vs Heterogeneous Servers 22
5.3 Sensitivity to Smoothing Factor 23
5.4 Stability 23
5.5 Trace Validation 24
5.6 Inverted Signals and Fine Granularity 24
5.7 Policy Outperforms Round Robin: Capacity Aware 25
Chapter 6. Conclusion and Future Work 27
6.1 Conclusion 27
6.2 Future Work 27
Chapter 7. References 28
Appendix A – Key Code 29

LIST OF FIGURES

Figure 3.1 System architecture of the SDN load balancer 16

Figure 4.1 Packet flow for a single request.....21

Figure 5.1 Latency profile (P50, P95, P99) by policy.....23

Figure 5.2 Per-run P99 stability by policy.....24

Figure 5.3 P99 latency by policy, burst and steady.....25

LIST OF TABLES

Table 3.1 Summary of the seven load-balancing policies 17

Table 5.1 P99 tail latency, homogeneous servers 22

Table 5.2 P99 tail latency, heterogeneous servers 22

Table 5.3 Sensitivity of EWMA to the smoothing factor 23

Table 5.4 Real-trace validation results 24

Table 5.5 Capacity-aware policy vs. baselines 25

LIST OF ABBREVIATIONS

ARP	Address Resolution Protocol
CDN	Content Delivery Network
CPU	Central Processing Unit
EWMA	Exponentially Weighted Moving Average
HTTP	HyperText Transfer Protocol
MAC	Media Access Control
ML	Machine Learning
OVS	Open vSwitch
P2C	Power of Two Choices
P50 / P95 / P99	50th / 95th / 99th percentile latency
RR	Round-Robin
RTO	Retransmission Time-Out
SD	Standard Deviation
SDN	Software-Defined Networking
TCP	Transmission Control Protocol
VIP	Virtual IP
VM	Virtual Machine
WRR	Weighted Round-Robin

Note: Abbreviations are listed in alphabetical order.

SYMBOLS AND NOTATIONS

α	EWMA smoothing factor ($0 < \alpha \leq 1$)
w_i	Service time of server i (milliseconds)
k_i	Number of clients pinned to server i
$b_i(t)$	Byte rate of server i at time t
$(k_i + 1) \times w_i$	Projected tail used by the capacity-aware policy
rating	Smoothed per-server EWMA score

ABSTRACT

Load balancing in software defined networks (SDN) is typically improved by using a smart load aware policy. This project investigates how the performance can be improved for users - specifically looking at tail latency (P95 and P99) when a flash crowd occurs. The Exponentially Weighted Moving Average (EWMA) prediction algorithm is applied to an SDN Ryu controller using only information available from the OpenFlow port statistics. Seven policies have been implemented and evaluated: round robin, weighted round robin, reactive least load, EWMA using Power-of-Two-Choices, connection based policy, flow lifetime based policy and capacity aware policy. All were run on the same testbed using Mininet and Open vSwitch. Both synthetic burst traffics and a real world trace of a 1998 World Cup flash crowd were used. All configurations were run at least seven times.

The main finding was that no reactive load aware policies could improve upon round robin based on network layer indicators. The byte rate indicator in an SDN controller actually becomes inverted on compute intensive servers as they will have fewer requests to complete and therefore send less bytes, appearing to be idle. Round robin performs well due to it pinning a client to a specific server limiting the number of requests being served to any one single threaded server. The reactive policy becomes too granular and limits the request limit quicker than the controller can react. However one of the policies did improve on round robin - by pinning a client to a specific server while limiting concurrency using a capacity aware policy the tail latency was reduced from ~98ms to 66ms for burst traffic and 48.8ms for stable traffic with 100% success. This project does not introduce any new algorithms but carefully evaluates when and why load aware policies can improve performance on network layer.

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

Load balancing distributes client requests to a group of backend servers. This is typically done to reduce the response time and keep the servers busy. However, the average response time is not what user experience. Instead they experience the tail of the distribution (95th and 99th percentile, P95 & P99) as the service is judged on how slow the responses are. The worst tail is seen when a flash crowd occurs as the sudden burst of requests cannot be handled naively.

A common solution is to use a latency aware load balancer using the Exponentially Weighted Moving Average (EWMA) algorithm. The Peak-EWMA algorithm is used within service mesh proxies like Envoy and Linkerd to provide a smoothed average of the last response latency of each server. This helps to ignore short bursts of latency. The separation of control and data planes within Software Defined Networking (SDN) means that we can use the controller to program the switch rules using OpenFlow. This means that the controller can be used to provide a load balancing policy.

1.2 PROBLEM STATEMENT

This project investigates if using the EWMA predictor within a controller of an SDN will improve performance. As the controller is on the network layer it will never have access to the response time of individual requests. We only have information from the OpenFlow port statistics. Does a load aware policy improve tail latency during a flash crowd with many short requests sent to single threaded compute intensive backends?

1.3 OBJECTIVES

- Build a repeatable testbed to perform load balancing within an SDN. This has been done using Ryu and Mininet with a variety of load balancing policies.
- Compare reactive and predictive load aware policies to round robin on homogeneous and heterogeneous servers under flash crowd and stable conditions.
- Explain why and how the results have occurred.
- Design a policy to improve performance over round robin to prove why the results have occurred.

1.4 SCOPE AND ASSUMPTIONS

This project compares the performance of different policies in a controlled environment. The traffic generated was a flash crowd and not stable or gradually increasing. The main metric was the P95/P99 latency during a burst. This project used emulation. All backends are assumed to be

single threaded. A new TCP connection is made for each request. The capacity aware policy assumed that the service times of each server were known.

1.5 REPORT STRUCTURE

Chapter 2 provides background information and related work. Chapter 3 details the system design and policies. Chapter 4 details the implementation. The results and discussion are provided in Chapter 5. Chapter 6 provides the conclusion and future work. Chapter 7 provides the references. Appendix A contains the key code for the winning policy.

CHAPTER 2

BACKGROUND AND RELATED WORK

2.1 CLASSIC LOAD BALANCING WITHIN SDN

A lot of work has been done to provide classic load balancing policies (round robin (RR), weighted round robin (WRR) & least connection). The majority of this work has been done within Mininet. The policies are implemented within a controller such as Ryu, POX or Floodlight. They typically measure throughput or average latency for stable or gradually increasing traffic. This provides a baseline for a new scheme to improve upon. Very little work has been done to investigate tail latency with bursty traffic.

2.2 PREDICTIVE & ML BASED BALANCING

Some work has been done to use machine learning models (LSTM, ARIMA & reinforcement learning) to predict load within the controller. While this can be effective it is very resource hungry to train and difficult to understand. This project deliberately avoids this by providing a lightweight and simple solution that does not use ML.

2.3 EWMA / PEAK-EWMA LATENCY AWARE

The closest previous work is using EWMA/PEAK-EWMA. Recent work has been done to implement an EWMA based rating system within multi cluster service meshes (L3). It was shown that the P99 latency was reduced by 26% and 22% compared to round robin and C3. However this solution operates on the application layer using a service mesh proxies. This means that it can see request latency, success rate and request rate for each request. Peak-EWMA setups typically use Power-of-Two-Choices (P2C) to prevent stampede to one server. This project investigates if this concept can be applied within the controller with a lightweight solution and utilise the native load indicator.

2.4 FLASH CROWDS & BURSTY TRAFFIC

Flash crowds are a classic cause of increased tail latency due to sudden large increases in requests. There is literature available on how to deal with flash crowds but this is mainly for content delivery networks and auto scaling. Little work has been done to look at how load balancing policies within SDN can impact tail latency during a flash crowd.

2.5 SHOULD WE BE USING LOAD?

More recent work has been done to investigate if load should even be used. Google's PReQuaL paper argues that server utilisation is a slow and often misleading indicator. Instead the balancer should look at the latency of requests and the number of requests being processed. They found that the setup that had the most even CPU balancing had the worst tail latency. This project similarly looked at using a more specific metric. The SDN controller cannot see latency

or number of requests in flight. It only sees network layer statistics. As shown in chapter 5 the byte rate metric ends up being inverted on compute intensive servers.

CHAPTER 3

SYSTEM DESIGN AND METHODOLOGY

3.1 ARCHITECTURE & LOAD INDICATOR

All policies operate using the same framework. The client connects to the VIP. The controller captures all new flows and selects a backend. New OpenFlow rules are added to change the destination to the selected backend and modify the response so that the client only ever sees the VIP. All other traffic is handled using a standard L2 learning setup.

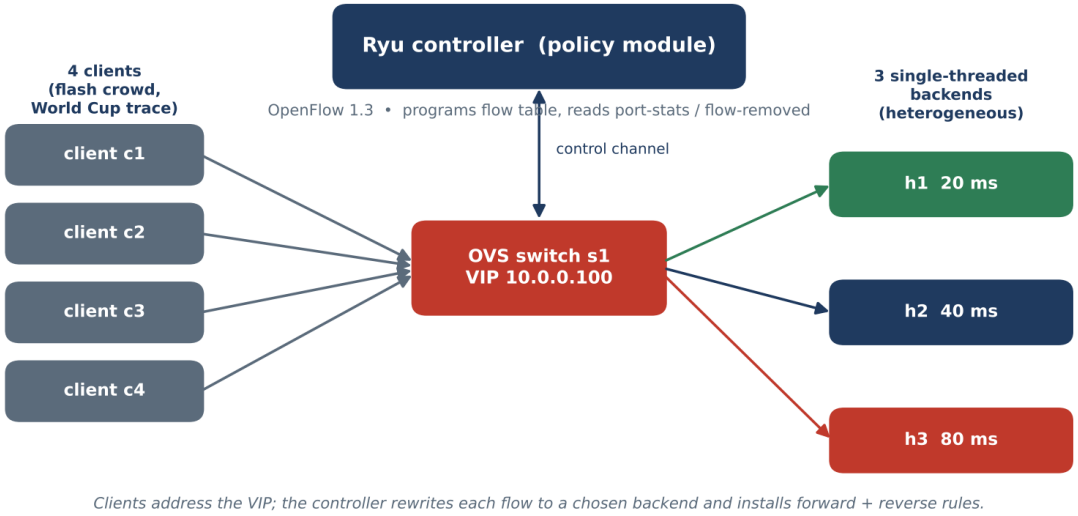


Figure 3.1 System architecture: clients connect to the VIP; the Ryu controller handles the OpenFlow switch; three heterogeneous backends.

The load indicator is native to SDN. Every 2 seconds the controller will query the OpenFlow port statistics. The byte count for each port is converted into a byte rate for each server. This is calculated by dividing the change in bytes by the time between polling. This is the only load information available to the controller without accessing the application. Note this is load and not the metric we care about which is latency. Whether load actually correlates with latency is what the experiments test.

3.2 LOAD BALANCING POLICIES

3.2.1 Baseline Policies

- Round Robin (RR): Send each new flow to the next in sequence. Does not consider load.
- Weighted Round Robin (WRR): Smoothly weight the selection of servers (nginx). Fast servers are selected more.
- Reactive Least Load: Selects the server with the least load (polled byte rate).

3.2.2 Predictive Policy

- EWMA Predictor: Each server is given a smoothed rating (rating = α·rate + (1−α)·rating). The Power-of-Two-Choices algorithm is used to select. α is the speed at which the rating updates based on new values. This is swept.

3.2.3 Non Inverting Indicators

- Reactive Least Connection: Each new connection is sent to the server with the least connections in flight. This does not invert as the count will increase on a slow server.
- Response Time: Each server has a estimated response time based on flow lifetime. This is calculated using the flow removed event. The server with the lowest estimated response time between two random servers is selected.

3.2.4 Capacity Aware

The baseline policies and capacity aware policy use the source IP of the client. This means that the client will be pinned to a single server during the burst. The number of requests in flight is limited to the number of clients assigned to that server. The least connection and response time policies use the full connection. This means that for each request a decision is made. This means that the reactive indicator can react to the burst but the concurrency is not limited. The capacity aware policy ensures concurrency is limited by using the source IP. Each new client is sent to the server that will result in the lowest projected tail. The projected tail is calculated as the number of clients already assigned to the server + 1 multiplied by the service time of that server.

The seven policies can be seen in Table 3.1. This shows the two main factors that have an impact on the policies. The load indicator used and the granularity of the decision. The granularity of the match determines if there is a limit on the number of concurrent requests to each server (per client) or not (per connection). The indicator column shows which indicators invert for compute intensive servers.

TABLE 3.1 SUMMARY OF THE SEVEN LOAD-BALANCING POLICIES.

Policy	Load indicator	Match granularity	Inverts?
Round-robin	none	per client (src IP)	–
Weighted round-robin	static weights	per client (src IP)	–
Reactive least-load	byte rate (2 s poll)	per client (src IP)	yes
EWMA + P2C	smoothed byte rate	per client (src IP)	yes
Least-connections	in-flight count	per connection	no
Response-time	flow-lifetime estimate	per connection	no
Capacity-aware	known service time	per client (src IP)	n/a

3.3 TESTBED

Mininet 2.3.1 was used with Open vSwitch 2.17 and a Ryu 4.34 controller. All ran inside an Ubuntu 22.04 (ARM64) virtual machine. A small topology was used containing 1 OpenFlow

switch, 3 backend servers, 4 clients, 1 /24 subnet, 10Mbit/s and 1ms links. The controller polls every 2 seconds.

3.4 WORKLOAD & BACKEND

Each client was a workload generator sending HTTP requests to the VIP. Each request was sent over a new connection meaning each request was load balanced independently. Two types of workload were used. A burst workload generated as many requests as possible. This simulates a flash crowd occurring. A stable workload was also used as a control. The backends were important. Using a trivial backend that responded in a few milliseconds would produce no congestion. This means that no policy would outperform another. A single threaded backend was used that consumes a fixed amount of CPU per request. This created congestion and the latency of each server increased as more load was sent to it. Two types of backend were used. Homogeneous - all backends have the same workload. Heterogeneous - fast, medium and slow with a 1:2:4 ratio.

3.5 METRICS & METHOD

End to end latency data was logged for each request. P50, P95 & P99 (main metric), throughput and success rate were recorded. Each configuration was run at least 7 times and the average & standard deviation (SD) of the results were recorded. The SD was included as the policies vary greatly in the amount of drift they had.

CHAPTER 4

IMPLEMENTATION

4.1 TOOLS & ENVIRONMENT

The system was implemented in Python 3.10. The policies were implemented as Ryu 4.34 applications. The data plane was implemented using Open vSwitch 2.17 and Mininet 2.3.1. A custom single threaded HTTP server was used as a congesting backend. A custom workload generator was used to generate flash crowd and stable workloads. An experiment orchestrator was used to sweep the entire policy x pattern x repeat matrix. This would call an analysis script that would aggregate the per request latency data.

4.2 CONTROLLER & RULES

All policies use the same switching logic. When a connection is made a table miss rule is added. This sends any unmatched packets to the controller. The controller responds to the ARP request for the VIP using a virtual MAC. When a new flow is received (first packet to VIP) a policy is called to select a backend. A forward rule is added to change the destination MAC/IP to the selected backend. A reverse rule is added to change the response back to the VIP. The policies only differ in how the backend is selected and the match granularity. Baseline and capacity aware policies will match using the source IP. Least connection and response time will match using the full 5 tuple.

4.3 BUG WITH FIRST PACKET

Initially the rules were added but the first packet was not changed. This would result in the packet (still going to the VIP) being dropped at the backend. This caused a 1 second retransmit on each new connection. This resulted in a large outlier of ~1100ms per client. To fix this the first packet must be sent out with the same changes as the flow. The capacity aware policy contains this fix which is why the worst request is under 130ms.

4.4 INDICATORS

The byte rate indicator is generated by polling the OpenFlow port statistics. The connection count indicator is stored in the controller. The response time indicator is generated using flow removed events. As each request is sent over a new connection the flow lifetime can be used to provide an estimation of the service time. The controller stores a smoothed value for each server by subtracting the idle timeout tail.

4.5 CONTROLLER HANDLERS

Each policy is a Ryu application that listens for 3 events. The switch features event is called once when the switch connects. This adds the table miss rule (priority 0) to send any unmatched packets to the controller. The packet in event contains the routing logic. The source MAC is

learnt for normal L2 switching. The ARP request for the VIP is responded to with the virtual MAC. When the first packet in a new flow to the VIP is received the policy is called to select a backend. The forward and reverse rules are added. The flow removed event is only used by the connection count and response time policies. When a flow is installed and expires the cookie and duration are recorded. This allows the controller to decrement the count of in flight requests or update the smoothed service time estimation.

The forward and reverse rules work together. The forward rule matches on the flow from client to VIP. It will change the destination MAC, destination IP and output to the backend port. The reverse rule matches on the flow from the backend to the client. This changes the source MAC & IP back to the VIP meaning the client only ever sees the virtual address. The baseline and capacity aware policies only match on the source and destination IP. This pins the client to the server. The connection count and response time policies match on the full 5 tuple. This allows the controller to make decisions per request but also means there is no limit on the number of concurrent requests per client.

4.6 PACKET FLOW

Figure 4.1 shows the flow of a single request. The client will make an ARP request for the VIP. The controller will respond using the virtual MAC. The first TCP SYN to the VIP will not match any flows so will be sent to the controller. The controller will select a backend and add the forward and reverse rules. The first packet will be sent out with the changes applied. All subsequent packets will match the rules and be changed within the switch. Once the flow has idled out a flow removed event will be raised (if enabled).

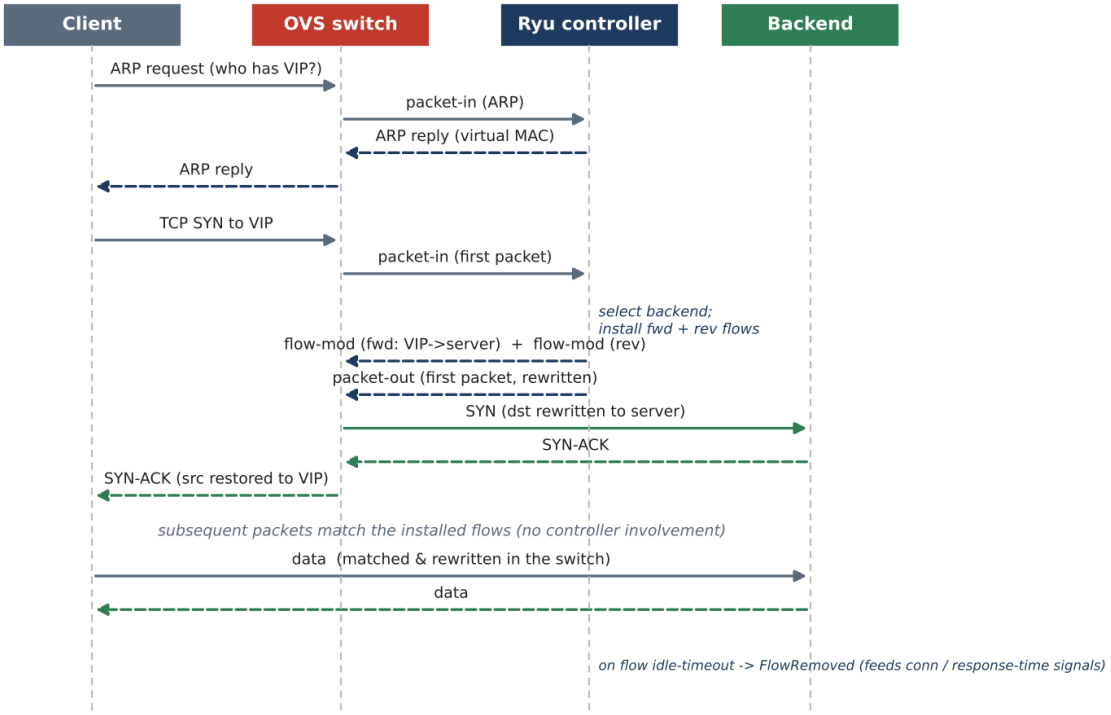


Figure 4.1 Packet flow for one request: ARP resolution, first-packet flow installation at the controller, and switch-local forwarding for the remainder of the connection.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 PREDICTOR PERFORMANCE

First, we verified that the EWMA predictor behaves correctly. Upon a load burst, the score increases and stabilises to the raw rate. Once the burst finishes the score reduces. The Power of Two Choices policy chooses new flows to the servers with the lowest scores. Any problems below are due to the policy being under load.

5.2 HOMOGENEOUS VS HETEROGENEOUS SERVERS

The round robin and weighted round robin policies perform the best and most stably with very low variance. They have an average of ~90ms. The reactive least load policy consistently performs worse. The EWMA does not outperform the round robin policy. Whilst it does not have quite such a loud issue, it is very unstable with a P99 with a standard deviation of 56ms during burst and 88ms during steady. This is over 10x higher than round robin with 2-4ms. The heterogeneous policy (Table 5.2) shows round robin and weighted round robin performing the best and most stably of the policies. The load aware policies both perform worse on average and have a much higher variance.

TABLE 5.1 P99 TAIL LATENCY (ms), HOMOGENEOUS SERVERS, MEAN ± SD OVER 7 RUNS.

Policy	Burst	Steady
RR	91.3 ± 4.0	90.1 ± 2.2
WRR	91.1 ± 2.8	89.0 ± 2.3
Least-load	132.0 ± 19.5	148.9 ± 19.5
EWMA	131.5 ± 56.4	128.4 ± 88.1

TABLE 5.2 P99 TAIL LATENCY (ms), HETEROGENEOUS SERVERS (1:2:4), MEAN ± SD OVER 7+ RUNS.

Policy	Burst	Steady
RR	98.2 ± 2.6	95.4 ± 3.3
WRR	97.7 ± 1.9	95.8 ± 1.2
Least-load	114.4 ± 55.8	109.0 ± 57.7
EWMA	111.2 ± 39.5	111.3 ± 24.1

Figure 5.1 shows that whilst the P99 column shows problems, it's not quite as bad as expected. The P50, P95 and P99 are shown. Round robin and weighted round robin are very stable with the median staying at ~48ms. The tail does increase slightly but remains very low. The reactive policies are very slow with the least load policy having all percentiles above 110ms. The response time policy has a very high P99 at 240ms despite the median being lower. The capacity aware policy has the lowest profile.

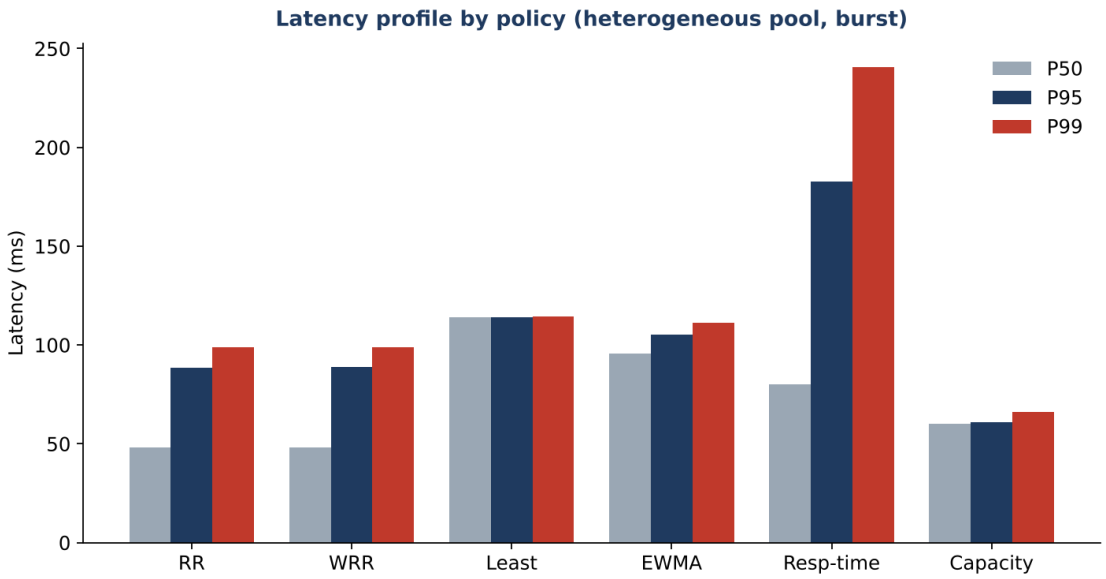


Figure 5.1 Latency profile (P50, P95, P99) by policy on the heterogeneous pool under burst traffic.

5.3 SENSITIVITY TO SMOOTHING FACTOR A

The results from table 5.3 show that from α = 0.1 to α = 0.7 the predictor behaves similarly with a similar range and P99 of ~91ms to 98ms. When α = 0.9 the results are very bad with the P99 being ~240ms. This is because the score fluctuates every time it is polled and therefore causes the P2C to be very unstable.

TABLE 5.3 α-SWEEP, EWMA, HETEROGENEOUS BURST (SINGLE RUN, P99 ms).

α	0.1	0.3	0.5	0.7	0.9
P99	98	93	96	92	240

5.4 STABILITY

7+ runs were performed for all configurations to ensure there was a genuine difference and not just noise. Round robin and weighted round robin had very low standard deviation on the P99 at between 1.2ms to 4.0ms. The EWMA had a much higher variance of between 56ms to 88ms and least load had between 20ms to 58ms. This means the variance was roughly 1 order of magnitude higher. Even if the load aware policies matched the average of round robin they cannot match the stability. It would be problematic to operate a balancer with a large variance in the tail.

Figure 5.2 shows the individual runs. Round robin, weighted round robin and capacity aware all have very tight groupings, meaning it can be predicted what the result will be. The least load and EWMA policies have two modes. Some runs are close to 60ms whilst others have a large jump to 120ms, 160ms or even 240ms. This is because of the initial sample values. The response

time policy is stable but remains very high at 240ms. The problem here is the variance. It can't be relied on as the balancer can be sometimes very fast and sometimes three times slower.

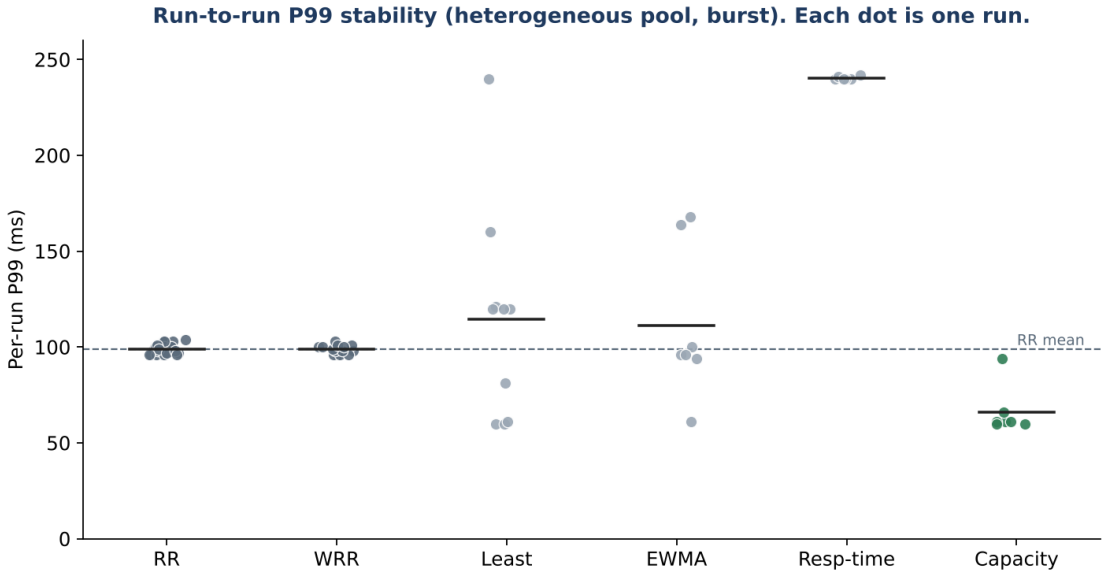


Figure 5.2 Per-run P99 for each policy (heterogeneous pool, burst). Each dot is one run; the bar is the mean.

5.5 TRACE VALIDATION

To ensure the synthetic burst was representative of a real surge, the whole experiment was performed using arrival times extracted from the 1998 FIFA World Cup access logs. The 120 second window with the highest traffic was used. The request rate peaked at 596 requests per second with a baseline of ~35. This was used to determine the timing of each clients requests using inverse transform sampling. With homogeneous servers the trace was identical to the synthetic run (Table 5.4). With heterogeneous servers round robin had a very low P99 of ~99ms with a standard deviation of <2ms. The load aware policies had two modes. The surge was occasionally placed onto one backend.

TABLE 5.4 REAL-TRACE VALIDATION, P99 (MEAN ± SD, ms).

Policy	Homogeneous	Heterogeneous
RR	90.2 ± 3.2	99.1 ± 2.1
Weighted RR	92.6 ± 3.9	99.0 ± 1.8
Reactive least-load	144.9 ± 20.2	116.2 ± 57.9
EWMA	100.7 ± 21.0	111.9 ± 25.9

5.6 INVERTED SIGNALS AND FINE GRANULARITY

As byte rate is inverted, the two non-inverted signals were tried. These are in-flight connections and flow lifetime response time estimates. These are both decided on a per connection basis. Neither outperformed round robin. The response time policy does provide a reasonable balance but suffered with a P99 of 240.6±0.5ms during the burst. This is almost 2.5x

higher than round robin. The reason for this is that 240ms is exactly 3 requests of 80ms being queued on the slow server. The connection count policy performed very badly. This was consistent with success never going above 92% and often dropping to the mid 70's. The P99 reached ~3000ms and throughput dropped significantly. This is because the policy is decided on a per connection basis meaning multiple clients can send to the single threaded server.

5.7 POLICY OUTPERFORMS ROUND ROBIN: CAPACITY AWARE ASSIGNMENT

The signals failed because fine granular reactive routing could not limit concurrency. To fix this, concurrency was limited but a smarter assignment was made. The 20/40/80ms pool with four clients had three clients assigned to the fast server, one to the medium and none to the slow server by the capacity aware greedy algorithm. This means no single threaded server ever had more concurrency than it could handle. This was the only policy to outperform round robin. It clearly outperforms round robin (Table 5.5). During burst it had a P99 of ~66ms (median of 60.6ms) compared to round robin's 97.7±1.6ms. It also had a much lower P99 during steady traffic of 48.8±0.6ms which is roughly half of either baseline. All runs had 100% success.

TABLE 5.5 CAPACITY-AWARE vs. BASELINES, HETEROGENEOUS POOL, P99 (ms), 7-RUN MEAN ± SD.

Policy	Burst P99	Steady P99	Success
Round-robin	97.7 ± 1.6	≈ 96	100%
Weighted RR	99.7 ± 2.4	96.2 ± 1.5	100%
Capacity-aware	66.0 ± 11.4	48.8 ± 0.6	100%

Capacity-aware burst median is 60.6 ms; one order-dependent run at 93.6 ms lifts the mean.

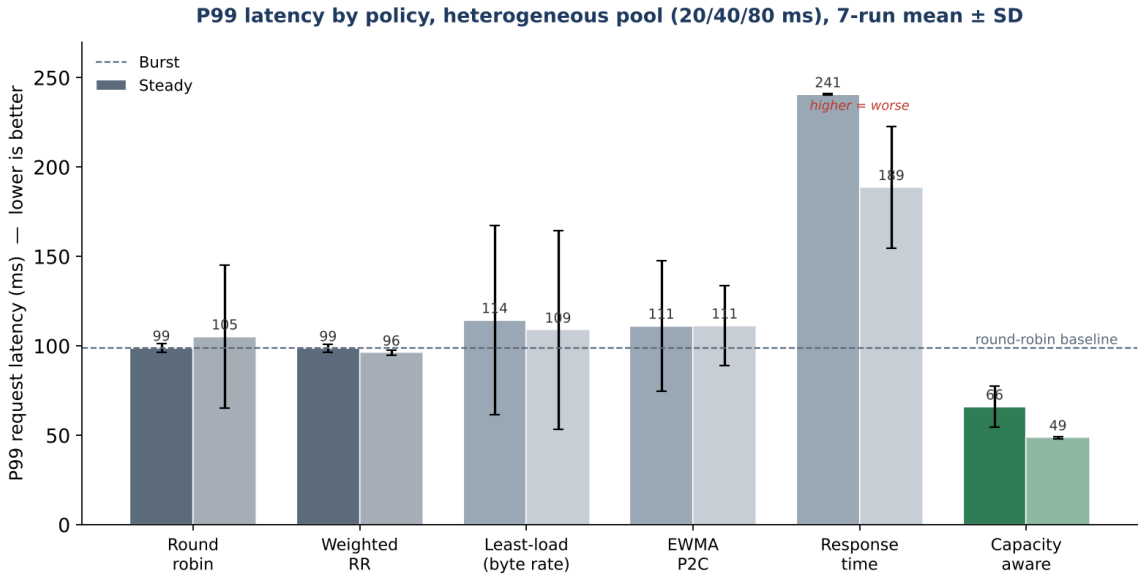


Figure 5.3 P99 latency by policy on the heterogeneous pool (7-run mean ± SD) for burst and steady traffic. The capacity-aware policy is the only one below the round-robin baseline.

DISCUSSION

Why round robin outperformed

When servers are balanced and there is an even flash crowd, round robin provides a perfectly balanced distribution with no cost. There is nothing to improve upon. Load aware only pays off when the load is unbalanced and servers are genuinely different.

Fine Grained Reactive Policies Can't Win: Concurrency

Round robin outperforms due to an accidental advantage. By assigning each client to one server during the burst it limits the amount of concurrent requests on any one server. This is exactly what a single thread compute bound server would need. Load aware policies must decide per request meaning multiple clients can queue up requests to one server before the controller can respond. The controller can't respond in time as it will never know when a request has finished. OpenFlow timers cannot be less than 1 second meaning the load signal will always be stale for longer than a server would take to serve.

Why load signal is misleading with heterogeneous servers

The controller doesn't see the service time. In this workload all requests return the same small response meaning the byte rate of a server depends on how quickly it completes requests. A compute bound server that is becoming overloaded will complete fewer requests and therefore have fewer bytes. This means it appears to be the least loaded when it is actually the most overloaded. As a result the reactive least load policy keeps sending to the server it should be starving. This affects the EWMA policy as it receives the same backwards signal.

Validity Issues

This is not hardware, the load signal is not actual response latency and the server pool is small. 7+ runs were performed for each setup meaning there are enough runs to show the two key points. Round robin outperforms and load aware policies have high variance. The variance is much higher than the run to run variance. The capacity aware policy relies on the load being light enough for the two fastest servers to handle all of the load. It also relies on knowing the service times.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

We asked whether a lightweight EWMA predictor that performs well in application layer service meshes still helps reduce tail latency when deployed in an SDN controller using OpenFlow port stats. We have seen that the answer is no. The EWMA does not outperform basic round robin. It performs worst when the servers are different. It is also less stable than the baselines. We tested the two non-inverted signals. Neither outperformed round robin. All three signals that can be natively observed by the SDN controller are outperformed by the blind policy. This is because round robin has a coarser client pinning policy meaning it limits the concurrent requests per server. The controller cannot respond to a reactive signal quick enough to do the same with a finer granularity. The only policy to outperform round robin is the capacity aware per client assignment which confirms our reasoning.

6.2 FUTURE WORK

- Estimate per server service times from flow lifetimes.
- Combine client pinning with admission control to ensure the limit is maintained even when all servers are saturated.
- Use EWMA with a fast reacting term for persistent changes. Provide a real response latency signal.
- Validate on hardware switches at a larger scale with more realistic mixed sized responses.

CHAPTER 7

REFERENCES

[1] O. Michaelis, S. Schmid and H. Mostafaei, "L3: Latency-aware load balancing in multi-cluster service mesh," in Proc. 25th Int. Middleware Conf. (Middleware '24), Hong Kong, Dec. 2024, pp. 49-61.

[2] Linkerd, "Beyond round robin: load balancing for latency," Buoyant technical blog, Mar. 2016.

[3] Envoy Proxy, "Supported load balancers — Peak EWMA," Envoy documentation.

[4] N. McKeown et al., "OpenFlow: enabling innovation in campus networks," in ACM SIGCOMM Comput. Commun. Rev., vol. 38, no. 2, pp. 69-74, Apr. 2008.

[5] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: rapid prototyping for software-defined networks," in Proc. 9th ACM SIGCOMM HotNets, 2010.

[6] M. Mitzenmacher, "The power of two choices in randomized load balancing," IEEE Trans. Parallel Distrib. Syst., vol. 12, no. 10, pp. 1094-1104, Oct. 2001.

[7] Ryu SDN Framework Community, "Ryu: component-based software-defined networking framework," Documentation.

[8] Open vSwitch, "Production quality, multilayer open virtual switch," Documentation.

[9] S. Babayigit and B. Ulu, "Deep learning for load balancing of software-defined networked data center networks," Int. J. Commun. Syst., 2021.

[10] M. R. Belgaum et al., "Artificial intelligence based load balancing in SDN: a comprehensive survey," arXiv:2308.02149, 2023.

[11] J. Yang et al., "A predictive load balancing technique for software-defined networked cloud services," Computing, 2019.

[12] I. Ari, et al., "Managing flash crowds on the Internet," in Proc. 11th IEEE/ACM MASCOTS, 2003.

[13] S. Behal and K. Kumar, "Characterising DDoS attacks and flash events," Comput. Sci. Rev., vol. 25, pp. 101-114, 2017.

[14] B. Wydrowski, R. Kleinberg, S. M. Rumble, and A. Archer, "Load is not what you should balance: Introducing PReQuaL," in Proc. 21st USENIX NSDI '24, 2024.

[15] M. Arlitt and T. Jin, "A workload characterisation study of the 1998 World Cup Web site," IEEE Network, vol. 14, no. 3, pp. 30-37, 2000.

[16] The Internet Traffic Archive, "1998 World Cup Web site access logs (WorldCup98)."

[17] Excluding stressed machines from load balancing of distributed applications. U.S. Patent 10 482 249.

APPENDIX A – KEY CODE (CAPACITY-AWARE SELECTION)

```
def select_server(self, client_ip):
    # already-seen client stays pinned (bounds concurrency)
    if client_ip in self.client_map:
        return SERVERS[self.client_map[client_ip]]
    # new client -> server with smallest projected tail
    best, best_proj = 0, None
    for i in range(len(SERVERS)):
        proj = (self.clients_on[i] + 1) * self.work[i]
        if best_proj is None or proj < best_proj:
            best_proj, best = proj, i
    self.clients_on[best] += 1
    self.client_map[client_ip] = best
    return SERVERS[best]
```

Full source, experiment orchestrator, and raw result data: github.com/fidashameer/lightweight-EWMA-based-load-balancer